

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 10%

QUESTIONS: 15

Total Marks:20

Annexure A

Comparative Analysis

Code of Conduct:

I Cheredla Jasvina/192324026 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Short Answer Test

1, Compare K-Nearest Neighbour and Case-Based Learning using a real-world example such as medical diagnosis. Analyze differences in memory usage, similarity measures, and prediction strategy.

Answer:

K-Nearest Neighbour (KNN) and Case-Based Learning (CBL) are both instance-based learning techniques. They make predictions by using previously stored examples instead of building a completely new mathematical model.

K-Nearest Neighbour (KNN): KNN stores the training data and classifies a new patient by finding the most similar patients in the database. For example, in medical diagnosis, patient information such as age, blood pressure, blood sugar, cholesterol, and symptoms can be used as features. If most of the nearest patients have a particular disease, the new patient is classified into that disease category.

Case-Based Learning (CBL): CBL stores previous complete cases and their solutions. For a new patient, the system searches for similar previous medical cases. It retrieves the relevant cases, reuses their diagnosis or treatment information, adapts it to the new patient, and can store the new case for future use.

Memory Usage: KNN generally stores the complete training dataset because it needs the stored feature values during prediction. Therefore, memory usage can become high when the number of patients is very large. CBL also requires memory to store previous cases, but it can store richer information such as symptoms, diagnosis, treatment, and outcome. Case-based reduction techniques can be used to remove unnecessary cases.

Similarity Measures: KNN usually uses mathematical distance measures such as Euclidean distance, Manhattan distance, or Minkowski distance. Before calculating distance, numerical features should normally be normalized. CBL can use more flexible similarity measures based on the application. In medical diagnosis, symptoms can be given different weights because some symptoms may be more important than others.

Prediction Strategy: KNN predicts using the majority class of the nearest patients. For example, if 4 out of 5 nearest patients have diabetes, the new patient may be classified as diabetic. CBL follows a more flexible process: **retrieve** similar cases, **reuse** their solutions, **revise** the solution according to the new patient, and **retain** the new case if it is useful.

Example: Suppose a new patient has high blood sugar, high cholesterol, increased thirst, and frequent urination. KNN searches for patients with similar numerical and categorical features and uses their majority diagnosis. CBL searches for previous patients with similar symptoms and medical histories and may examine what diagnosis and treatment worked for those patients before making a recommendation.

Main Differences:

Feature	KNN	Case-Based Learning
Memory usage	Stores training examples	Stores complete previous cases
Similarity	Mainly distance-based	Can use domain-specific similarity
Prediction	Majority voting among neighbours	Retrieves and adapts previous solutions
Adaptability	Lower	Higher
Training	Almost no explicit training	Almost no traditional training
Explanation	Based on nearest examples	Based on similar previous cases

Conclusion: KNN is simpler and mainly performs classification using the nearest examples, while Case-Based Learning is more flexible because it can retrieve, adapt, revise, and retain complete previous cases. In medical diagnosis, KNN is useful for straightforward disease classification, whereas CBL is useful when detailed previous medical cases and treatment outcomes are available.

2. Compare Locally Weighted Regression and Radial Basis Function networks for function approximation tasks. Use an example dataset to explain differences in locality, computational cost, and smoothness of predictions.

Answer:

Locally Weighted Regression (LWR) and Radial Basis Function (RBF) networks are machine learning techniques used for function approximation. Both methods can model nonlinear relationships, but they differ in how they use local information, their computational requirements, and the smoothness of their predictions.

Locally Weighted Regression (LWR): LWR gives greater importance to training points that are close to the input being predicted and lower importance to points that are far away. Instead of learning one global model, LWR builds a local regression model around each query point. For example, consider a dataset containing temperature and electricity consumption:

Temperature = 20, 22, 25, 28, 30, 32

Electricity Consumption = 40, 45, 52, 65, 75, 90

If we want to predict electricity consumption at a temperature of 27°C, LWR gives higher weights to the observations at 25°C and 28°C because they are close to 27°C. The observations at 20°C and 32°C receive smaller weights.

A commonly used weighting function is:

where x is the query point, x_i is a training point, and k controls the size of the local region.

Radial Basis Function Network (RBF): An RBF network uses radial basis functions, usually Gaussian functions, to represent the relationship between input and output. Each hidden neuron has a center, and its activation depends on the distance between the input and that center.

The Gaussian radial basis function can be written as:

where μ is the center of the radial basis function and σ represents its width.

For the same temperature example, an RBF network may learn centers around temperatures such as 22°C, 27°C, and 31°C. When the input is 27°C, the RBF centered near 27°C produces a strong response.

Locality: LWR calculates locality separately for every new input. For each prediction, it determines which training points are close to the query and assigns weights accordingly.

Therefore, LWR is highly local and adapts its regression model to the current input.

RBF networks also use locality, but the local regions are represented by learned radial basis functions. Once the centers and widths have been learned, the same basis functions are used for future predictions.

Computational Cost: LWR has relatively little training because it does not build a fixed global model. However, prediction can be expensive because every new input may require calculating distances, weights, and a local regression model.

RBF networks usually require more computation during training because the centers, widths, and output weights must be learned. However, once trained, prediction is generally faster because the system only evaluates the fixed number of RBF units.

Smoothness of Predictions: LWR can produce smooth predictions when an appropriate weighting function and bandwidth are selected. If the bandwidth is very small, predictions may change rapidly and become sensitive to individual data points. If the bandwidth is large, predictions become smoother but may lose important local details.

RBF networks generally produce smooth nonlinear functions because Gaussian basis functions overlap with one another. The smoothness depends mainly on the number of basis functions and their widths.

Difference in Adaptability: LWR can adapt strongly to local changes in the dataset because it calculates a new local model for each query. RBF networks have a fixed learned structure after training, so they are less directly adaptive to new patterns unless the network is retrained.

Example: Suppose the temperature–electricity relationship is nonlinear. At low temperatures, electricity usage may be moderate, while at high temperatures it may increase rapidly because of air-conditioning usage. LWR can focus on nearby temperatures and accurately model this local change. An RBF network can learn several Gaussian regions across the temperature range and combine them to create a smooth approximation of the complete relationship.

Advantages of LWR: It is simple, flexible, highly local, and can model nonlinear relationships without requiring a complex global model.

Advantages of RBF: It provides smooth nonlinear approximation and can make predictions efficiently after the network has been trained.

Conclusion: LWR is more strongly query-dependent and locally adaptive, while an RBF network learns a fixed set of local basis functions during training. LWR usually has lower training cost but higher prediction cost, whereas RBF networks have higher training cost but generally faster prediction. For a small dataset where local adaptation is important, LWR can be useful. For repeated predictions and smooth function approximation, an RBF network is often more suitable.

3. Analyze the differences between K-Nearest Neighbour and Radial Basis Function models in classification problems. Provide an example and compare decision boundaries, training time, and generalization ability.

Answer:

K-Nearest Neighbour (KNN) and Radial Basis Function (RBF) models are both used for classification problems. Both methods use the concept of similarity or distance between data points, but they work differently. KNN makes predictions directly from stored training examples, while an RBF model learns a mathematical representation of the data during training.

K-Nearest Neighbour (KNN): KNN is a supervised learning algorithm that stores the training data. When a new sample is given, it calculates the distance between the new sample and the stored training samples. It selects the closest samples and assigns the class based on majority voting.

For example, consider a customer classification system that predicts whether a customer is **Low Risk** or **High Risk** using income and credit score. If , KNN finds the five customers closest to the new customer. If four are classified as Low Risk and one is High Risk, the new customer is classified as Low Risk.

Radial Basis Function (RBF) Model: An RBF model uses a neural network structure with an input layer, a hidden layer containing radial basis functions, and an output layer. The hidden neurons measure the distance between an input and their learned centers.

A common Gaussian RBF function is:

where x is the input, c is the center of the RBF unit, and σ controls the width of the function.

The outputs of the RBF units are combined to determine the final class.

Decision Boundaries: KNN can create highly irregular and local decision boundaries. The shape of the boundary depends strongly on the value of k . With a small k , such as 1, the boundary can be very complex because individual training points have a strong influence. Increasing k generally makes the boundary smoother.

RBF models usually produce smoother decision boundaries because several radial basis functions overlap and their outputs are combined. The centers and widths determine the shape of the classification regions.

For example, if Low Risk and High Risk customers are separated by a nonlinear boundary, KNN can follow the local structure directly, while an RBF model can learn a smooth nonlinear boundary around the two classes.

Training Time: KNN has almost no traditional training stage. It mainly stores the training data, so the training process is very fast. However, the computational cost is shifted to the prediction stage.

RBF models require more training. The centers of the radial basis functions must be selected, their widths must be determined, and the output weights must be learned. Therefore, training an RBF model takes more time than storing data for KNN.

Prediction Time: KNN can be slow for large datasets because a new sample may need to be compared with many stored training samples. Efficient search structures can reduce this cost. Once an RBF model has been trained, prediction is usually faster because the input only needs to be passed through a fixed number of radial basis functions.

Generalization Ability: KNN can generalize well when the training data is representative and an appropriate value of k is selected. However, a very small k can cause overfitting because the model becomes sensitive to noise and individual training samples. A larger k can improve robustness but may oversmooth the classification boundary.

RBF models can provide good generalization because they learn smooth nonlinear relationships. However, generalization depends on the number and location of RBF centers, the widths of the basis functions, and the amount of training data. Too many narrow basis functions can cause overfitting, while too few or very broad basis functions can cause underfitting.

Memory Usage: KNN requires storing the training examples, so memory usage can become large for big datasets. RBF models store the learned centers, widths, and output weights rather than the complete training dataset, which can reduce memory requirements after training.

Example: Suppose a bank wants to classify customers as **Approved** or **Rejected** based on income and credit score. KNN can classify a new customer by looking at nearby customers. An RBF classifier can learn several centers representing different customer groups and use these learned regions to determine the class.

Advantages of KNN: KNN is simple, easy to understand, requires almost no training, and can model complex decision boundaries directly from the data.

Advantages of RBF: RBF models provide smooth nonlinear decision boundaries, can make predictions quickly after training, and generally require less memory during prediction.

Limitations of KNN: Prediction can be computationally expensive for large datasets, and performance can decrease in high-dimensional spaces because distance becomes less meaningful.

Limitations of RBF: Training is more complicated than KNN, and selecting suitable centers and widths is important for achieving good performance.

Conclusion: KNN is preferable when simplicity, fast training, and direct use of training examples are important. RBF models are preferable when smooth nonlinear decision boundaries, faster repeated prediction, and a compact learned model are required.